

GSOI 2026 夏季常规赛

第十试题解

Ehundategh & tfbz

Gioush 大队

2026 年 7 月

目录

T1 员题 (race)

T2 翻转字符 (flip)

T3 战争 (war)

T4 我们最好的最短路 (shortest)

吐槽时间

Contents

T1 员题 (race)

T2 翻转字符 (flip)

T3 战争 (war)

T4 我们最好的最短路 (shortest)

吐槽时间

数据点 1 ~ 2 20 Pts

- ▶ 一种比赛安排就是 Ehundategh 的 n 匹马与对手的 n 匹马之间的一个双射。直接枚举全部 $n!$ 个双射，再计算每种安排的总收益。

数据点 1 ~ 2 20 Pts

- ▶ 一种比赛安排就是 Ehundategh 的 n 匹马与对手的 n 匹马之间的一个双射。直接枚举全部 $n!$ 个双射，再计算每种安排的总收益。
- ▶ 每种安排需要 $\mathcal{O}(n)$ 的时间检查，因此时间复杂度为 $\mathcal{O}(n!n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 1 ~ 2 20 Pts

- ▶ 一种比赛安排就是 Ehundategh 的 n 匹马与对手的 n 匹马之间的一个双射。直接枚举全部 $n!$ 个双射，再计算每种安排的总收益。
- ▶ 每种安排需要 $\mathcal{O}(n)$ 的时间检查，因此时间复杂度为 $\mathcal{O}(n!n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 枚举的瓶颈在于同时决定全部配对。若能证明某一场比赛一定可以出现在最优方案中，就可以每次固定一场比赛。

数据点 3 ~ 4 40 Pts

- ▶ 先用插入排序将双方的马按速度升序排列，再维护双方尚未参赛的最慢马与最快马，执行正解中的两端贪心。

数据点 3 ~ 4 40 Pts

- ▶ 先用插入排序将双方的马按速度升序排列，再维护双方尚未参赛的最慢马与最快马，执行正解中的两端贪心。
- ▶ 贪心扫描只需要 $\mathcal{O}(n)$ 的时间，这一档的瓶颈变成了插入排序。时间复杂度为 $\mathcal{O}(n^2)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 3 ~ 4 40 Pts

- ▶ 先用插入排序将双方的马按速度升序排列，再维护双方尚未参赛的最慢马与最快马，执行正解中的两端贪心。
- ▶ 贪心扫描只需要 $\mathcal{O}(n)$ 的时间，这一档的瓶颈变成了插入排序。时间复杂度为 $\mathcal{O}(n^2)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 数据范围扩大后，贪心过程不需要改变，只需将排序替换为 $\mathcal{O}(n \log n)$ 的比较排序。

正解 100 Pts

- ▶ 将 Ehundategh 的速度序列记为 a ，对手的速度序列记为 b ，分别升序排序。维护 l_a, r_a, l_b, r_b ，表示双方尚未参赛的最慢马与最快马。

正解 100 Pts

- ▶ 将 Ehundategh 的速度序列记为 a ，对手的速度序列记为 b ，分别升序排序。维护 l_a, r_a, l_b, r_b ，表示双方尚未参赛的最慢马与最快马。
- ▶ 若 $a_{r_a} > b_{r_b}$ ，让两匹最快马比赛。否则，若 $a_{l_a} > b_{l_b}$ ，让两匹最慢马比赛。

正解 100 Pts

- ▶ 将 Ehundategh 的速度序列记为 a ，对手的速度序列记为 b ，分别升序排序。维护 l_a, r_a, l_b, r_b ，表示双方尚未参赛的最慢马与最快马。
- ▶ 若 $a_{r_a} > b_{r_b}$ ，让两匹最快马比赛。否则，若 $a_{l_a} > b_{l_b}$ ，让两匹最慢马比赛。
- ▶ 两种胜局都不存在时，让 a_{l_a} 与 b_{r_b} 比赛。若 $a_{l_a} < b_{r_b}$ ，收益为 -1 ，否则收益为 0 。

正解 100 Pts

- 先考虑 $a_{r_a} > b_{r_b}$ 。在某个最优方案中，设 a_{r_a} 对阵 b_j ，另一匹马 a_k 对阵 b_{r_b} 。

正解 100 Pts

- ▶ 先考虑 $a_{r_a} > b_{r_b}$ 。在某个最优方案中, 设 a_{r_a} 对阵 b_j , 另一匹马 a_k 对阵 b_{r_b} 。
- ▶ 交换这两场的对手后, a_{r_a} 仍然获胜。又因为 $b_j \leq b_{r_b}$, 所以 a_k 对阵 b_j 的结果不会比对阵 b_{r_b} 更差。

正解 100 Pts

- ▶ 先考虑 $a_{r_a} > b_{r_b}$ 。在某个最优方案中，设 a_{r_a} 对阵 b_j ，另一匹马 a_k 对阵 b_{r_b} 。
- ▶ 交换这两场的对手后， a_{r_a} 仍然获胜。又因为 $b_j \leq b_{r_b}$ ，所以 a_k 对阵 b_j 的结果不会比对阵 b_{r_b} 更差。
- ▶ 因此存在一个最优方案让两匹最快马比赛，可以直接固定这一场胜局。

正解 100 Pts

- 再考虑 $a_{r_a} \leq b_{r_b}$ 且 $a_{l_a} > b_{l_b}$ 。设最优方案中 a_{l_a} 对阵 b_j , a_k 对阵 b_{l_b} 。

正解 100 Pts

- ▶ 再考虑 $a_{r_a} \leq b_{r_b}$ 且 $a_{l_a} > b_{l_b}$ 。设最优方案中 a_{l_a} 对阵 b_j , a_k 对阵 b_{l_b} 。
- ▶ 交换后, a_{l_a} 可以战胜 b_{l_b} 。由于 $a_k \geq a_{l_a}$, a_k 对阵 b_j 的结果不会比 a_{l_a} 对阵 b_j 更差。

正解 100 Pts

- ▶ 再考虑 $a_{r_a} \leq b_{r_b}$ 且 $a_{l_a} > b_{l_b}$ 。设最优方案中 a_{l_a} 对阵 b_j , a_k 对阵 b_{l_b} 。
- ▶ 交换后, a_{l_a} 可以战胜 b_{l_b} 。由于 $a_k \geq a_{l_a}$, a_k 对阵 b_j 的结果不会比 a_{l_a} 对阵 b_j 更差。
- ▶ 因此也存在一个最优方案让两匹最慢马比赛, 这一场胜局同样可以固定。

正解 100 Pts

- 最后只剩 $a_{r_a} \leq b_{r_b}$ 且 $a_{l_a} \leq b_{l_b}$ 。此时 a_{l_a} 无法战胜任何剩余的对手， b_{r_b} 也不会输给任何剩余的马。

正解 100 Pts

- ▶ 最后只剩 $a_{r_a} \leq b_{r_b}$ 且 $a_{l_a} \leq b_{l_b}$ 。此时 a_{l_a} 无法战胜任何剩余的对手, b_{r_b} 也不会输给任何剩余的马。
- ▶ 设某个最优方案中 a_{l_a} 对阵 b_j , a_k 对阵 b_{r_b} 。原来两场的收益都只能是 -1 或 0 , 现在交换这两场的对手。

正解 100 Pts

- ▶ 最后只剩 $a_{r_a} \leq b_{r_b}$ 且 $a_{l_a} \leq b_{l_b}$ 。此时 a_{l_a} 无法战胜任何剩余的对手, b_{r_b} 也不会输给任何剩余的马。
- ▶ 设某个最优方案中 a_{l_a} 对阵 b_j , a_k 对阵 b_{r_b} 。原来两场的收益都只能是 -1 或 0 , 现在交换这两场的对手。
- ▶ 若原来两场都失败, 交换后显然不会比 -2 更差。若恰有一场平局, 交换后 a_k 对阵更慢的 b_j 至少平局, 另一场至多损失 1 , 总收益仍不下降。

正解 100 Pts

- 若原来两场都是平局，则 $a_{l_a} = b_j$ 且 $a_k = b_{r_b}$ 。若四匹马速度相同，交换后仍是两场平局。

正解 100 Pts

- ▶ 若原来两场都是平局，则 $a_{l_a} = b_j$ 且 $a_k = b_{r_b}$ 。若四匹马速度相同，交换后仍是两场平局。
- ▶ 否则 $a_{l_a} < b_{r_b}$ 且 $a_k > b_j$ ，交换后恰好得到一负一胜，总收益仍为 0。

正解 100 Pts

- ▶ 若原来两场都是平局，则 $a_{l_a} = b_j$ 且 $a_k = b_{r_b}$ 。若四匹马速度相同，交换后仍是两场平局。
- ▶ 否则 $a_{l_a} < b_{r_b}$ 且 $a_k > b_j$ ，交换后恰好得到一负一胜，总收益仍为 0。
- ▶ 四种情况均不会使总收益下降，所以可以用最慢马消耗对方最快马。每一步都固定了某个最优方案中的一场比赛，重复这一过程即可得到全局最优解。

正解 100 Pts

- ▶ 排序需要 $\mathcal{O}(n \log n)$ 的时间，四个指针只会单向移动，扫描需要 $\mathcal{O}(n)$ 的时间。

正解 100 Pts

- ▶ 排序需要 $\mathcal{O}(n \log n)$ 的时间，四个指针只会单向移动，扫描需要 $\mathcal{O}(n)$ 的时间。
- ▶ 总时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。答案位于 $[-n, n]$ ，使用 `int` 即可。

正解 100 Pts

- ▶ 排序需要 $\mathcal{O}(n \log n)$ 的时间，四个指针只会单向移动，扫描需要 $\mathcal{O}(n)$ 的时间。
- ▶ 总时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。答案位于 $[-n, n]$ ，使用 `int` 即可。
- ▶ 实现时速度相同的比赛收益必须为 0。第三种情况只有在 $a_{l_a} < b_{r_b}$ 时才令答案减 1。

正解 100 Pts

```
1  scanf("%d",&c);
2  scanf("%d",&n);
3  for (int i=1;i<=n;i++) scanf("%d",&LineB[i]);
4  for (int i=1;i<=n;i++) scanf("%d",&LineA[i]);
5  sort(LineA+1,LineA+n+1);
6  sort(LineB+1,LineB+n+1);
7  int LeftA=1,RightA=n,LeftB=1,RightB=n,Ans=0;
```

正解 100 Pts

```
1  while (LeftA<=RightA) {
2      if (LineA[RightA]>LineB[RightB]) {
3          Ans++;
4          RightA--;RightB--;
5      }
6      else if (LineA[LeftA]>LineB[LeftB]) {
7          Ans++;
8          LeftA++;LeftB++;
9      }
10     else {
11         if (LineA[LeftA]<LineB[RightB]) Ans--;
12         LeftA++;RightB--;
13     }
14 }
15 printf("%d\n",Ans);
16 return 0;
17 }
```

Contents

T1 员题 (race)

T2 翻转字符 (flip)

T3 战争 (war)

T4 我们最好的最短路 (shortest)

吐槽时间

数据点 1 ~ 2 20 Pts

- ▶ 记翻转后的初始字符串为 B 。用集合 S 表示已经从 B 中取出并依次放到目标串前缀的位置，令 f_S 为完成这些位置所需的最少交换次数。

数据点 1 ~ 2 20 Pts

- ▶ 记翻转后的初始字符串为 B 。用集合 S 表示已经从 B 中取出并依次放到目标串前缀的位置，令 f_S 为完成这些位置所需的最少交换次数。
- ▶ 设 $k = |S|$ 。下一步选择一个 $p \notin S$ ，要求 $B_p = A_{k+1}$ 。把 B_p 移到所有未取出字符的最前面，需要跨过它左侧仍未取出的字符。

数据点 1 ~ 2 20 Pts

- ▶ 记翻转后的初始字符串为 B 。用集合 S 表示已经从 B 中取出并依次放到目标串前缀的位置，令 f_S 为完成这些位置所需的最少交换次数。
- ▶ 设 $k = |S|$ 。下一步选择一个 $p \notin S$ ，要求 $B_p = A_{k+1}$ 。把 B_p 移到所有未取出字符的最前面，需要跨过它左侧仍未取出的字符。
- ▶ 初始化 $f_\emptyset = 0$ ，其余状态为正无穷。答案为 $f_{\{1,2,\dots,n\}}$ 。

数据点 1 ~ 2 20 Pts

► 完整转移为

$$f_{S \cup \{p\}} = \min \left\{ f_{S \cup \{p\}}, f_S + |\{j \mid 1 \leq j < p, j \notin S\}| \right\},$$

其中 $p \notin S$ 且 $B_p = A_{|S|+1}$ 。

数据点 1 ~ 2 20 Pts

完整转移为

$$f_{S \cup \{p\}} = \min \left\{ f_{S \cup \{p\}}, f_S + |\{j \mid 1 \leq j < p, j \notin S\}| \right\},$$

其中 $p \notin S$ 且 $B_p = A_{|S|+1}$ 。

- 按集合大小从小到大转移。状态数为 2^n ，每个状态枚举位置并计算跨过的字符数，时间复杂度为 $\mathcal{O}(n^2 2^n)$ ，空间复杂度为 $\mathcal{O}(2^n)$ 。

数据点 3 ~ 5 50 Pts

- ▶ 直接维护当前字符串。从左到右处理目标位置 i ，在当前位置之后找到第一个等于 A_i 的字符，再通过相邻交换把它移动到位置 i 。

数据点 3 ~ 5 50 Pts

- ▶ 直接维护当前字符串。从左到右处理目标位置 i ，在当前位置之后找到第一个等于 A_i 的字符，再通过相邻交换把它移动到位置 i 。
- ▶ 若这个字符当前位于位置 j ，将它移动到位置 i 需要 $j-i$ 次相邻交换。它依次越过前方的 $j-i$ 个字符，恰好改变了这 $j-i$ 对字符的相对顺序。

数据点 3 ~ 5 50 Pts

- ▶ 直接维护当前字符串。从左到右处理目标位置 i ，在当前位置之后找到第一个等于 A_i 的字符，再通过相邻交换把它移动到位置 i 。
- ▶ 若这个字符当前位于位置 j ，将它移动到位置 i 需要 $j-i$ 次相邻交换。它依次越过前方的 $j-i$ 个字符，恰好改变了这 $j-i$ 对字符的相对顺序。
- ▶ 将重复字符按这次选择的先后顺序编号后，整个过程就是对一个位置排列执行冒泡排序，交换次数等于排列的逆序对数量。

数据点 3 ~ 5 50 Pts

- ▶ 直接维护当前字符串。从左到右处理目标位置 i ，在当前位置之后找到第一个等于 A_i 的字符，再通过相邻交换把它移动到位置 i 。
- ▶ 若这个字符当前位于位置 j ，将它移动到位置 i 需要 $j-i$ 次相邻交换。它依次越过前方的 $j-i$ 个字符，恰好改变了这 $j-i$ 对字符的相对顺序。
- ▶ 将重复字符按这次选择的先后顺序编号后，整个过程就是对一个位置排列执行冒泡排序，交换次数等于排列的逆序对数量。
- ▶ 模拟需要 $\mathcal{O}(n^2)$ 的时间。接下来只需证明重复字符应该怎样对应，再快速统计这个位置排列的逆序对。

正解 100 Pts

- 固定一种字符 x 。设它在目标串 A 中依次出现于 $p_1 < p_2 < \dots < p_k$ ，在初始串 B 中依次出现于 $q_1 < q_2 < \dots < q_k$ 。

正解 100 Pts

- ▶ 固定一种字符 x 。设它在目标串 A 中依次出现于 $p_1 < p_2 < \dots < p_k$ ，在初始串 B 中依次出现于 $q_1 < q_2 < \dots < q_k$ 。
- ▶ 若 $q_i < q_j$ 却分别对应 $p_s > p_t$ ，交换两个相同字符的目标位置不会改变最终字符串，并且会消去这一次交叉。

正解 100 Pts

- ▶ 固定一种字符 x 。设它在目标串 A 中依次出现于 $p_1 < p_2 < \dots < p_k$ ，在初始串 B 中依次出现于 $q_1 < q_2 < \dots < q_k$ 。
- ▶ 若 $q_i < q_j$ 却分别对应 $p_s > p_t$ ，交换两个相同字符的目标位置不会改变最终字符串，并且会消去这一次交叉。
- ▶ 对任意第三个目标位置 r ，若 r 不在 p_t, p_s 之间，交换前后的逆序关系数量不变。若 $p_t < r < p_s$ ，交换会再消去两处逆序。因此交换不会增加代价。

正解 100 Pts

- ▶ 固定一种字符 x 。设它在目标串 A 中依次出现于 $p_1 < p_2 < \dots < p_k$ ，在初始串 B 中依次出现于 $q_1 < q_2 < \dots < q_k$ 。
- ▶ 若 $q_i < q_j$ 却分别对应 $p_s > p_t$ ，交换两个相同字符的目标位置不会改变最终字符串，并且会消去这一次交叉。
- ▶ 对任意第三个目标位置 r ，若 r 不在 p_t, p_s 之间，交换前后的逆序关系数量不变。若 $p_t < r < p_s$ ，交换会再消去两处逆序。因此交换不会增加代价。
- ▶ 反复交换全部交叉关系，就能得到 q_i 对应 p_i 的稳定匹配，并且某个最优方案一定采用这种对应。

正解 100 Pts

- 按 B 中字符的顺序，依次取出该字符在 A 中尚未使用的最早位置，得到一个排列 $P_1, P_2, \dots, P_n$ 。

正解 100 Pts

- ▶ 按 B 中字符的顺序，依次取出该字符在 A 中尚未使用的最早位置，得到一个排列 $P_1, P_2, \dots, P_n$ 。
- ▶ 上一档中，一个字符向左移动多少格，就会消去多少个逆序对。若 $i < j$ 且 $P_i > P_j$ ，这两个字符在 B 与 A 中的相对顺序不同，二者至少进行一次相邻交换。

正解 100 Pts

- ▶ 按 B 中字符的顺序, 依次取出该字符在 A 中尚未使用的最早位置, 得到一个排列 $P_1, P_2, \dots, P_n$ 。
- ▶ 上一档中, 一个字符向左移动多少格, 就会消去多少个逆序对。若 $i < j$ 且 $P_i > P_j$, 这两个字符在 B 与 A 中的相对顺序不同, 二者至少进行一次相邻交换。
- ▶ 因此答案至少是排列的逆序对数量。另一方面, 按照冒泡排序逐一消去逆序对, 每次交换都使逆序对数量减少 1, 最终恰好得到目标顺序。

正解 100 Pts

- ▶ 按 B 中字符的顺序, 依次取出该字符在 A 中尚未使用的最早位置, 得到一个排列 $P_1, P_2, \dots, P_n$ 。
- ▶ 上一档中, 一个字符向左移动多少格, 就会消去多少个逆序对。若 $i < j$ 且 $P_i > P_j$, 这两个字符在 B 与 A 中的相对顺序不同, 二者至少进行一次相邻交换。
- ▶ 因此答案至少是排列的逆序对数量。另一方面, 按照冒泡排序逐一消去逆序对, 每次交换都使逆序对数量减少 1, 最终恰好得到目标顺序。
- ▶ 所以最少相邻交换次数恰好等于排列的逆序对数量, 正解只需要在不模拟交换的情况下统计它。

正解 100 Pts

- ▶ 我们不需要显式构造 B 。先把 A 中每种字符的出现位置从左到右放入队列，再倒序扫描 A ，这正好等价于从左到右扫描 B 。

正解 100 Pts

- ▶ 我们不需要显式构造 B 。先把 A 中每种字符的出现位置从左到右放入队列，再倒序扫描 A ，这正好等价于从左到右扫描 B 。
- ▶ 每遇到一个字符，就弹出对应队列的队首位置，并依次写入 Line 。例如 $A = \text{aaaza}$ 时得到的排列是 $(1, 4, 2, 3, 5)$ ，逆序对数量为 2。

正解 100 Pts

- ▶ 我们不需要显式构造 B 。先把 A 中每种字符的出现位置从左到右放入队列，再倒序扫描 A ，这正好等价于从左到右扫描 B 。
- ▶ 每遇到一个字符，就弹出对应队列的队首位置，并依次写入 Line 。例如 $A = \text{aaaza}$ 时得到的排列是 $(1, 4, 2, 3, 5)$ ，逆序对数量为 2。
- ▶ 扫描 Line ，查询此前加入且大于当前位置的元素数量，再把当前位置加入线段树。

正解 100 Pts

- ▶ 线段树维护每个位置是否已经出现，支持单点修改与区间求和。每个位置执行一次查询与一次修改。

正解 100 Pts

- ▶ 线段树维护每个位置是否已经出现，支持单点修改与区间求和。每个位置执行一次查询与一次修改。
- ▶ 时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

- ▶ 线段树维护每个位置是否已经出现，支持单点修改与区间求和。每个位置执行一次查询与一次修改。
- ▶ 时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 答案最大为 $\frac{n(n-1)}{2}$ ，必须使用 long long。同一字符必须弹出最早尚未使用的位置，任意匹配会破坏稳定对应。

正解 100 Pts

```
1 void Modify(int Now,int l,int r,int x) {
2     if (l==r) {
3         Tree[Now]=1;
4         return;
5     }
6     int Mid=(l+r)>>1;
7     if (x<=Mid) Modify(Now<<1,l,Mid,x);
8     else Modify(Now<<1|1,Mid+1,r,x);
9     Tree[Now]=Tree[Now<<1]+Tree[Now<<1|1];
10 }
11
12 int Query(int Now,int l,int r,int x,int y) {
13     if (x>y||l>y||r<x) return 0;
14     if (l>=x&&r<=y) return Tree[Now];
15     int Mid=(l+r)>>1;
16     return Query(Now<<1,l,Mid,x,y)+Query(Now<<1|1,Mid+1,r,x,y);
17 }
```

正解 100 Pts

```
1  scanf("%d",&c);
2  scanf("%d%s",&n,Str+1);
3  for (int i=1;i<=n;i++) Pos[Str[i]-'a'].push(i);
4  for (int i=n;i>=1;i--) {
5      int x=Str[i]-'a';
6      Line[n-i+1]=Pos[x].front();
7      Pos[x].pop();
8  }
```

正解 100 Pts

```
1  long long Ans=0;
2  for (int i=1;i<=n;i++) {
3      Ans+=Query(1,1,n,Line[i]+1,n);
4      Modify(1,1,n,Line[i]);
5  }
6  printf("%lld\n",Ans);
7  return 0;
8  }
```


Contents

T1 员题 (race)

T2 翻转字符 (flip)

T3 战争 (war)

T4 我们最好的最短路 (shortest)

吐槽时间

数据点 1 ~ 6 15 Pts

- ▶ 对每个并查集根维护两个位集合， $\text{Member}(u)$ 表示联盟中的成员， $\text{Enemy}(u)$ 表示这些成员的全部敌对对象。

数据点 1 ~ 6 15 Pts

- ▶ 对每个并查集根维护两个位集合， $\text{Member}(u)$ 表示联盟中的成员， $\text{Enemy}(u)$ 表示这些成员的全部敌对对象。
- ▶ 合并 x, y 前，检查 $\text{Enemy}(x) \cap \text{Member}(y)$ 是否为空。接受提议后，将两类位集合分别按位或。

数据点 1 ~ 6 15 Pts

- ▶ 对每个并查集根维护两个位集合， $\text{Member}(u)$ 表示联盟中的成员， $\text{Enemy}(u)$ 表示这些成员的全部敌对对象。
- ▶ 合并 x, y 前，检查 $\text{Enemy}(x) \cap \text{Member}(y)$ 是否为空。接受提议后，将两类位集合分别按位或。
- ▶ 每次提议进行常数长度次长度为 n 的位运算，时间复杂度为 $\mathcal{O}((n+q)n/w)$ ，其中 w 为机器字长，空间复杂度为 $\mathcal{O}(n^2/w)$ 。

数据点 7 ~ 12 17 Pts

- ▶ 这一档满足 $m \leq 250$ 。并查集维护当前联盟，每次提议扫描全部初始敌对关系。

数据点 7 ~ 12 17 Pts

- ▶ 这一档满足 $m \leq 250$ 。并查集维护当前联盟，每次提议扫描全部初始敌对关系。
- ▶ 对一条敌对关系 (u, v) ，若 $\text{Find}(u)$ 与 $\text{Find}(v)$ 分别等于待合并的两个根，这次提议必须拒绝。若全部敌对关系都没有跨越两个联盟，才能执行合并。

数据点 7 ~ 12 17 Pts

- ▶ 这一档满足 $m \leq 250$ 。并查集维护当前联盟，每次提议扫描全部初始敌对关系。
- ▶ 对一条敌对关系 (u, v) ，若 $\text{Find}(u)$ 与 $\text{Find}(v)$ 分别等于待合并的两个根，这次提议必须拒绝。若全部敌对关系都没有跨越两个联盟，才能执行合并。
- ▶ 时间复杂度为 $\mathcal{O}(qm\alpha(n))$ ，空间复杂度为 $\mathcal{O}(n + m)$ 。这一做法的瓶颈是每次重复扫描全部敌对关系。

数据点 13 ~ 18 20 Pts

- 这一档满足 $n, m \leq 5 \times 10^3$ ，继续使用第一档的位集合方法。

数据点 13 ~ 18 20 Pts

- ▶ 这一档满足 $n, m \leq 5 \times 10^3$ ，继续使用第一档的位集合方法。
- ▶ 位集合把一次冲突检查与一次联盟信息合并都压缩成 $\mathcal{O}(n/w)$ 个机器字操作，足以处理 $q \leq 10^5$ 。

数据点 13 ~ 18 20 Pts

- ▶ 这一档满足 $n, m \leq 5 \times 10^3$ ，继续使用第一档的位集合方法。
- ▶ 位集合把一次冲突检查与一次联盟信息合并都压缩成 $\mathcal{O}(n/w)$ 个机器字操作，足以处理 $q \leq 10^5$ 。
- ▶ 与第一档相同，时间复杂度为 $\mathcal{O}((n+q)n/w)$ ，空间复杂度为 $\mathcal{O}(n^2/w)$ 。当 n 扩大到 10^5 时，二维位集合的空间无法承受。

数据点 19 ~ 24 23 Pts

- ▶ 这一档保证至多有一次提议被拒绝。暂时接受全部提议，每次合并建立一个权值等于当前提议编号的新结点，得到并查集重构森林。

数据点 19 ~ 24 23 Pts

- ▶ 这一档保证至多有一次提议被拒绝。暂时接受全部提议，每次合并建立一个权值等于当前提议编号的新结点，得到并查集重构森林。
- ▶ 一对参与者首次进入同一联盟的时刻，恰好是它们在重构树上最近公共祖先的权值。枚举全部敌对关系，取这些时刻的最小值，得到唯一可能被拒绝的提议。

数据点 19 ~ 24 23 Pts

- ▶ 这一档保证至多有一次提议被拒绝。暂时接受全部提议，每次合并建立一个权值等于当前提议编号的新结点，得到并查集重构森林。
- ▶ 一对参与者首次进入同一联盟的时刻，恰好是它们在重构树上最近公共祖先的权值。枚举全部敌对关系，取这些时刻的最小值，得到唯一可能被拒绝的提议。
- ▶ 跳过这一提议，其余提议根据特殊性质都能被接受。时间复杂度为 $\mathcal{O}((n + m + q) \log n)$ ，空间复杂度为 $\mathcal{O}((n + q) \log n)$ 。

正解 100 Pts

- ▶ 并查集维护当前联盟。对于每个根 u ，维护集合 $\mathcal{A}_u$ ，其中保存与联盟 u 之间至少存在一对初始敌对参与者的其他联盟根。

正解 100 Pts

- ▶ 并查集维护当前联盟。对于每个根 u ，维护集合 $\mathcal{A}_u$ ，其中保存与联盟 u 之间至少存在一对初始敌对参与者的其他联盟根。
- ▶ 始终维持三个不变量： $\mathcal{A}_u$ 中只出现当前根，敌对关系双向存在，并且 $v \in \mathcal{A}_u$ 当且仅当联盟 u, v 之间存在敌对参与者。

正解 100 Pts

- ▶ 并查集维护当前联盟。对于每个根 u ，维护集合 $\mathcal{A}_u$ ，其中保存与联盟 u 之间至少存在一对初始敌对参与者的其他联盟根。
- ▶ 始终维持三个不变量： $\mathcal{A}_u$ 中只出现当前根，敌对关系双向存在，并且 $v \in \mathcal{A}_u$ 当且仅当联盟 u, v 之间存在敌对参与者。
- ▶ 初始时每个人单独形成联盟，每条敌对关系同时加入两端的集合，所以三个不变量成立。

正解 100 Pts

- ▶ 处理提议 (x, y) 时，先把二者替换为当前并查集根。若 $x = y$ ，两个参与者已经在同一联盟中，直接输出 Yes。

正解 100 Pts

- ▶ 处理提议 (x, y) 时，先把二者替换为当前并查集根。若 $x = y$ ，两个参与者已经在同一联盟中，直接输出 Yes。
- ▶ 若 $y \in \mathcal{A}_x$ ，两个联盟之间存在敌对参与者，输出 No，并且不能修改任何结构。

正解 100 Pts

- ▶ 处理提议 (x, y) 时, 先把二者替换为当前并查集根。若 $x = y$, 两个参与者已经在同一联盟中, 直接输出 Yes。
- ▶ 若 $y \in \mathcal{A}_x$, 两个联盟之间存在敌对参与者, 输出 No, 并且不能修改任何结构。
- ▶ 剩余情况可以安全合并。判断能否合并只需要在集合中查找另一个根, 不再扫描全部初始敌对关系。

正解 100 Pts

- ▶ 设较小联盟的根为 x ，较大联盟的根为 y 。枚举每个 $v \in \mathcal{A}_x$ ，在 $\mathcal{A}_v$ 中删除 x 并加入 y ，同时把 v 加入 $\mathcal{A}_y$ 。

正解 100 Pts

- ▶ 设较小联盟的根为 x ，较大联盟的根为 y 。枚举每个 $v \in \mathcal{A}_x$ ，在 $\mathcal{A}_v$ 中删除 x 并加入 y ，同时把 v 加入 $\mathcal{A}_y$ 。
- ▶ 完成全部关系迁移后，清空 $\mathcal{A}_x$ ，令 x 的并查集父亲为 y ，并更新联盟大小。

正解 100 Pts

- ▶ 设较小联盟的根为 x ，较大联盟的根为 y 。枚举每个 $v \in \mathcal{A}_x$ ，在 $\mathcal{A}_v$ 中删除 x 并加入 y ，同时把 v 加入 $\mathcal{A}_y$ 。
- ▶ 完成全部关系迁移后，清空 $\mathcal{A}_x$ ，令 x 的并查集父亲为 y ，并更新联盟大小。
- ▶ 这一步把所有原来连接到 x 的敌对关系改接到 y 。集合会自动消除重复元素，因此三个不变量在合并后继续成立。

正解 100 Pts

- ▶ 每次只迁移较小联盟的敌对集合。一个集合元素被迁移后，它所依附的联盟大小至少翻倍，所以每条敌对关系至多被迁移 $\mathcal{O}(\log n)$ 次。

正解 100 Pts

- ▶ 每次只迁移较小联盟的敌对集合。一个集合元素被迁移后，它所依附的联盟大小至少翻倍，所以每条敌对关系至多被迁移 $\mathcal{O}(\log n)$ 次。
- ▶ 每次 set 查找、插入与删除需要 $\mathcal{O}(\log n)$ 的时间。总时间复杂度为

$$\mathcal{O}(m \log^2 n + q \log n + (n + m + q)\alpha(n)).$$

正解 100 Pts

- ▶ 每次只迁移较小联盟的敌对集合。一个集合元素被迁移后，它所依附的联盟大小至少翻倍，所以每条敌对关系至多被迁移 $\mathcal{O}(\log n)$ 次。
- ▶ 每次 set 查找、插入与删除需要 $\mathcal{O}(\log n)$ 的时间。总时间复杂度为

$$\mathcal{O}(m \log^2 n + q \log n + (n + m + q)\alpha(n)).$$

- ▶ 空间复杂度为 $\mathcal{O}(n + m)$ 。实现时必须先处理 $x = y$ ，拒绝提议时也不能合并并查集。

正解 100 Pts

```
1 int Find(int x) {
2     return Fa[x]==x?x:Fa[x]=Find(Fa[x]);
3 }
4
5 void Merge(int x,int y) {
6     if (Size[x]>Size[y]) swap(x,y);
7     for (set<int>::iterator it=Against[x].begin();it!=Against[x].end();
8         it++) {
9         int To=Find(*it);
10        Against[To].erase(x);
11        Against[To].insert(y);
12        Against[y].insert(To);
13    }
14    Against[x].clear();
15    Fa[x]=y;
16    Size[y]+=Size[x];
17 }
```

正解 100 Pts

```
1  scanf("%d",&c);
2  scanf("%d%d%d",&n,&m,&q);
3  for (int i=1;i<=n;i++) Fa[i]=i,Size[i]=1;
4  for (int i=1;i<=m;i++) {
5      int x,y;
6      scanf("%d%d",&x,&y);
7      Against[x].insert(y);
8      Against[y].insert(x);
9  }
```

正解 100 Pts

```
1  while (q-->0) {
2      int x,y;
3      scanf("%d%d",&x,&y);
4      x=Find(x);y=Find(y);
5      if (x==y) {
6          puts("Yes");
7          continue;
8      }
9      if (Against[x].find(y)!=Against[x].end()) {
10         puts("No");
11         continue;
12     }
13     Merge(x,y);
14     puts("Yes");
15 }
16 return 0;
17 }
```

Contents

T1 员题 (race)

T2 翻转字符 (flip)

T3 战争 (war)

T4 我们最好的最短路 (shortest)

吐槽时间

数据点 1 ~ 2 20 Pts

- ▶ 当 $n \leq 300$ 时，使用 Floyd 算法预处理任意两点之间的最短路，随后每次询问直接输出答案。

数据点 1 ~ 2 20 Pts

- ▶ 当 $n \leq 300$ 时, 使用 Floyd 算法预处理任意两点之间的最短路, 随后每次询问直接输出答案。
- ▶ 令 $d_{i,j}$ 表示当前已允许若干中转点时, 从 i 到 j 的最短距离。依次加入中转点 k , 执行

$$d_{i,j} \leftarrow \min\{d_{i,j}, d_{i,k} + d_{k,j}\}.$$

数据点 1 ~ 2 20 Pts

- ▶ 当 $n \leq 300$ 时, 使用 Floyd 算法预处理任意两点之间的最短路, 随后每次询问直接输出答案。
- ▶ 令 $d_{i,j}$ 表示当前已允许若干中转点时, 从 i 到 j 的最短距离。依次加入中转点 k , 执行

$$d_{i,j} \leftarrow \min\{d_{i,j}, d_{i,k} + d_{k,j}\}.$$

- ▶ 时间复杂度为 $\mathcal{O}(n^3 + q)$, 空间复杂度为 $\mathcal{O}(n^2)$ 。

数据点 3 ~ 4 20 Pts

- ▶ 特殊性质 A 保证原图是一棵树。任选结点 1 为根，倍增预处理最近公共祖先，并记根到 u 的距离为 d_u 。

数据点 3 ~ 4 20 Pts

- ▶ 特殊性质 A 保证原图是一棵树。任选结点 1 为根，倍增预处理最近公共祖先，并记根到 u 的距离为 d_u 。
- ▶ 树上两点之间只有一条简单路径，因此

$$\text{dist}_T(u, v) = d_u + d_v - 2d_{\text{LCA}(u, v)}.$$

数据点 3 ~ 4 20 Pts

- ▶ 特殊性质 A 保证原图是一棵树。任选结点 1 为根，倍增预处理最近公共祖先，并记根到 u 的距离为 d_u 。
- ▶ 树上两点之间只有一条简单路径，因此

$$\text{dist}_T(u, v) = d_u + d_v - 2d_{\text{LCA}(u, v)}.$$

- ▶ 时间复杂度为 $\mathcal{O}((n + q) \log n)$ ，空间复杂度为 $\mathcal{O}(n \log n)$ 。

数据点 5 ~ 6 20 Pts

- ▶ 这一档满足 $q \leq 300$ 。对每次询问从起点运行一次堆优化 Dijkstra，再读取终点的最短距离。

数据点 5 ~ 6 20 Pts

- ▶ 这一档满足 $q \leq 300$ 。对每次询问从起点运行一次堆优化 Dijkstra，再读取终点的最短距离。
- ▶ 每次询问都重新计算整张图上的单源最短路，时间复杂度为 $\mathcal{O}(qm \log n)$ ，空间复杂度为 $\mathcal{O}(n + m)$ 。

数据点 5 ~ 6 20 Pts

- ▶ 这一档满足 $q \leq 300$ 。对每次询问从起点运行一次堆优化 Dijkstra，再读取终点的最短距离。
- ▶ 每次询问都重新计算整张图上的单源最短路，时间复杂度为 $\mathcal{O}(qm \log n)$ ，空间复杂度为 $\mathcal{O}(n + m)$ 。
- ▶ 当询问数扩大到 10^5 时，瓶颈在于重复运行 Dijkstra。需要找到一组数量很少、能够覆盖所有非树最短路的固定起点。

正解 100 Pts

- 从原图中任取一棵生成树 T 。生成树有 $n - 1$ 条边，所以树外边数量为

$$m - (n - 1) = m - n + 1 \leq 21.$$

正解 100 Pts

- 从原图中任取一棵生成树 T 。生成树有 $n - 1$ 条边，所以树外边数量为

$$m - (n - 1) = m - n + 1 \leq 21.$$

- 将所有树外边的两个端点放入集合 K ，则 $|K| \leq 42$ 。任意使用过树外边的路径都一定经过某个 $k \in K$ 。

正解 100 Pts

- 从原图中任取一棵生成树 T 。生成树有 $n - 1$ 条边，所以树外边数量为

$$m - (n - 1) = m - n + 1 \leq 21.$$

- 将所有树外边的两个端点放入集合 K ，则 $|K| \leq 42$ 。任意使用过树外边的路径都一定经过某个 $k \in K$ 。
- 这正是上一档所需的少量固定起点。只需要从每个关键点运行一次 Dijkstra，而不是为每次询问重新运行。

正解 100 Pts

- ▶ 构造生成树时，可以按输入顺序使用并查集选择树边。若一条边连接两个不同连通块，就加入生成树，否则它是树外边，并记录两个端点。

正解 100 Pts

- ▶ 构造生成树时，可以按输入顺序使用并查集选择树边。若一条边连接两个不同连通块，就加入生成树，否则它是树外边，并记录两个端点。
- ▶ 生成树不需要是最小生成树。树的作用只是提供一条始终合法的候选路径，并把所有其余边压缩到至多 21 条。

正解 100 Pts

- ▶ 构造生成树时，可以按输入顺序使用并查集选择树边。若一条边连接两个不同连通块，就加入生成树，否则它是树外边，并记录两个端点。
- ▶ 生成树不需要是最小生成树。树的作用只是提供一条始终合法的候选路径，并把所有其余边压缩到至多 21 条。
- ▶ 在生成树上预处理深度、倍增祖先与根距离，从而支持 $\text{dist}_T(u, v)$ 。再从每个 $k \in K$ 在原图上运行 Dijkstra，记结果为 $\text{dist}_k(v)$ 。

正解 100 Pts

► 对于一次询问 (u, v) , 答案为

$$\min \left\{ \text{dist}_T(u, v), \min_{k \in K} (\text{dist}_k(u) + \text{dist}_k(v)) \right\}.$$

正解 100 Pts

- 对于一次询问 (u, v) , 答案为

$$\min \left\{ \text{dist}_T(u, v), \min_{k \in K} (\text{dist}_k(u) + \text{dist}_k(v)) \right\}.$$

- 第一项对应完全在生成树上行走。第二项枚举一个关键点, 并把 u 到 k 与 k 到 v 的两条最短路拼接起来。

正解 100 Pts

- 对于一次询问 (u, v) , 答案为

$$\min \left\{ \text{dist}_T(u, v), \min_{k \in K} (\text{dist}_k(u) + \text{dist}_k(v)) \right\}.$$

- 第一项对应完全在生成树上行走。第二项枚举一个关键点，并把 u 到 k 与 k 到 v 的两条最短路拼接起来。
- 每一项都对应原图中的一条合法路线，所以这些候选值都不会小于真正的最短路。

正解 100 Pts

- 取一条从 u 到 v 的最短路 P 。若 P 不经过树外边，那么 P 完全位于生成树中，只能是树上的唯一简单路径，第一项恰好等于答案。

正解 100 Pts

- ▶ 取一条从 u 到 v 的最短路 P 。若 P 不经过树外边，那么 P 完全位于生成树中，只能是树上的唯一简单路径，第一项恰好等于答案。
- ▶ 若 P 经过树外边，任选这条路径上的一个树外边端点 k 。将 P 在 k 处分成两段，可以得到

$$\text{dist}_k(u) + \text{dist}_k(v) \leq \text{length}(P).$$

正解 100 Pts

- ▶ 取一条从 u 到 v 的最短路 P 。若 P 不经过树外边，那么 P 完全位于生成树中，只能是树上的唯一简单路径，第一项恰好等于答案。
- ▶ 若 P 经过树外边，任选这条路径上的一个树外边端点 k 。将 P 在 k 处分成两段，可以得到

$$\text{dist}_k(u) + \text{dist}_k(v) \leq \text{length}(P).$$

- ▶ 左侧本身又是一条从 u 经过 k 到 v 的合法路线，所以它不可能小于 $\text{length}(P)$ ，故二者相等。

正解 100 Pts

- ▶ 上一页说明任意最短路都能被公式中的某一项取得，结合所有候选均为合法路线，公式的必要性与充分性均已成立。

正解 100 Pts

- ▶ 上一页说明任意最短路都能被公式中的某一项取得，结合所有候选均为合法路线，公式的必要性与充分性均已成立。
- ▶ 预处理生成树需要 $\mathcal{O}(n \log n)$ 的时间，从至多 42 个关键点运行 Dijkstra 需要 $\mathcal{O}(|K|m \log n)$ 的时间。

正解 100 Pts

- ▶ 上一页说明任意最短路都能被公式中的某一项取得，结合所有候选均为合法路线，公式的必要性与充分性均已成立。
- ▶ 预处理生成树需要 $\mathcal{O}(n \log n)$ 的时间，从至多 42 个关键点运行 Dijkstra 需要 $\mathcal{O}(|K|m \log n)$ 的时间。
- ▶ 每次询问需要一次树上距离和 $|K|$ 个关键点候选。总时间复杂度为

$$\mathcal{O}(n \log n + |K|m \log n + q(|K| + \log n)),$$

空间复杂度为 $\mathcal{O}(n \log n + |K|n + m)$ 。

正解 100 Pts

- ▶ 边权最大为 10^9 ，路径长度和 Dijkstra 距离必须使用 long long。关键点需要去重，否则只会重复运行相同的 Dijkstra。

正解 100 Pts

- ▶ 边权最大为 10^9 ，路径长度和 Dijkstra 距离必须使用 long long。关键点需要去重，否则只会重复运行相同的 Dijkstra。
- ▶ 实现时用 Dis 保存生成树根距离，用 Dist[i][v] 保存第 i 个关键点到 v 的原图最短距离。

正解 100 Pts

- ▶ 边权最大为 10^9 ，路径长度和 Dijkstra 距离必须使用 long long。关键点需要去重，否则只会重复运行相同的 Dijkstra。
- ▶ 实现时用 Dis 保存生成树根距离，用 Dist[i][v] 保存第 i 个关键点到 v 的原图最短距离。
- ▶ 构造生成树时仍需把每条边加入原图，只有生成树邻接结构与关键点集合根据并查集结果分别维护。

正解 100 Pts

```
1 void Edge_Add(int St,int Ed,int Val){Edge[++Total]={Ed,Head[St],Val};  
   Head[St]=Total;}  
2 void Tree_Add(int St,int Ed,int Val){Tree[++TreeTotal]={Ed,TreeHead[St],  
   Val};TreeHead[St]=TreeTotal;}  
3 int Find(int x){return SetFa[x]==x?x:SetFa[x]=Find(SetFa[x]);}  
4 void Insert(int x){if (!Mark[x]) Mark[x]=1,Key[++cnt]=x;}  
5  
6 void Build() {  
7     queue <int> Q;  
8     Dep[1]=1;Q.push(1);  
9     while (Q.size()) {  
10         int Now=Q.front();Q.pop();
```

正解 100 Pts

```
1      for (int i=TreeHead[Now];i;i=Tree[i].Next) {  
2          int To=Tree[i].Ed;  
3          if (To==Fa[Now][0]) continue;  
4          Fa[To][0]=Now;Dep[To]=Dep[Now]+1;  
5          Dis[To]=Dis[Now]+Tree[i].Val;  
6          for (int j=1;j<=21;j++) Fa[To][j]=Fa[Fa[To][j-1]][j-1];  
7          Q.push(To);  
8      }  
9  }  
10 }
```

正解 100 Pts

```
1 int LCA(int x,int y) {
2     if (Dep[x]<Dep[y]) swap(x,y);
3     for (int i=21;i>=0;i--) if (Dep[Fa[x][i]]>=Dep[y]) x=Fa[x][i];
4     if (x==y) return x;
5     for (int i=21;i>=0;i--) {
6         if (Fa[x][i]!=Fa[y][i]) x=Fa[x][i],y=Fa[y][i];
7     }
8     return Fa[x][0];
9 }
10
11 long long Query(int x,int y) {
12     int lca=LCA(x,y);
13     return Dis[x]+Dis[y]-2*Dis[lca];
14 }
```

正解 100 Pts

```
1 void Dijkstra(int x,int S) {
2     memset(Dist[x],0x3f,sizeof(Dist[x]));
3     Dist[x][S]=0;Q.emplace(0,S);
4     while (Q.size()) {
5         long long Value=Q.top().first;
6         int Now=Q.top().second;Q.pop();
7         if (Value!=Dist[x][Now]) continue;
8         for (int i=Head[Now];i;i=Edge[i].Next) {
9             int To=Edge[i].Ed;
10            if (Dist[x][To]>Dist[x][Now]+Edge[i].Val) {
11                Dist[x][To]=Dist[x][Now]+Edge[i].Val;
12                Q.emplace(Dist[x][To],To);
13            }
14        }
15    }
16 }
```

正解 100 Pts

```
1  scanf("%d",&c);
2  scanf("%d%d",&n,&m);
3  for (int i=1;i<=n;i++) SetFa[i]=i,Size[i]=1;
4  for (int i=1;i<=m;i++) {
5      scanf("%d%d%d",&In1,&In2,&In3);
6      Edge_Add(In1,In2,In3);Edge_Add(In2,In1,In3);
7      int x=Find(In1),y=Find(In2);
8      if (x!=y) {
9          if (Size[x]>Size[y]) swap(x,y);
10         SetFa[x]=y;Size[y]+=Size[x];
11         Tree_Add(In1,In2,In3);Tree_Add(In2,In1,In3);
12     }
13     else Insert(In1),Insert(In2);
14 }
15 Build();
16 for (int i=1;i<=cnt;i++) Dijkstra(i,Key[i]);
```

正解 100 Pts

```
1  scanf("%d",&q);
2  while (q-->0) {
3      scanf("%d%d",&In1,&In2);
4      long long Ans=Query(In1,In2);
5      for (int i=1;i<=cnt;i++) Ans=min(Ans,Dist[i][In1]+Dist[i][In2]);
6      printf("%lld\n",Ans);
7  }
8  return 0;
9  }
```

Contents

T1 员题 (race)

T2 翻转字符 (flip)

T3 战争 (war)

T4 我们最好的最短路 (shortest)

吐槽时间